FICHA DE ACOMPANHAMENTO – APS 2026

Universidade Paulista – UNIP | Ciência da Computação – 7º e 8º semestres
Disciplina: Desenvolvimento de Sistemas Distribuídos
Etapa 5 – Experimentos de escalabilidade e resiliência

1. IDENTIFICAÇÃO

Grupo nº / Turma		**Data**	
Domínio da aplicação			
Repositório de código			
Ambiente de medição	Máquina do cluster: CPU _____ núcleos ____ RAM ____ GB | Cluster: _____ nós ____ | Gerador na mesma máquina? ( ) Sim ( ) Não		
Serviço mais exigido			

2. INTEGRANTES DO GRUPO

#	NOME COMPLETO	RA	RESPONSABILIDADE NESTA ETAPA
1			
2			
3			
4			
5			

3. OBJETIVO DA ETAPA

Executar, com o gerador de carga da Etapa 4, os **cinco experimentos obrigatórios restantes**: escalabilidade horizontal manual (1, 2, 4 e 8 réplicas), escalabilidade automática (HPA), resiliência e autorrecuperação (pod apagado e nó drenado), atualização sem interrupção (*rolling update* e *rollout undo*) e efeito do cache ou do processamento assíncrono – cada um com hipótese, procedimento, evidências, tabela, gráfico e análise crítica.
(Documento da APS 2026, item II.4 – Experimentos obrigatórios; cronograma II.6, itens 6 e 7.)

4. CHECKLIST DOS REQUISITOS OBRIGATÓRIOS

Preenchimento pelo grupo; conferência pelo professor durante a demonstração.

#	REQUISITO	EVIDÊNCIA / COMANDO	SIM	NÃO
1	**Escala manual**: 1, 2, 4 e 8 réplicas, mesma carga, HPA removido	`kubectl scale` + 4 resumos	( )	( )
2	Ganho em relação a 1 réplica calculado e comparado com o ideal	tabela 5.1 + gráfico	( )	( )
3	**HPA** por CPU (mín. 2, máx. 10) com carga em rampa	`kubectl get hpa -w`	( )	( )
4	Linha do tempo do HPA: carga, decisão, pods prontos, latência normal	eventos com hora + série	( )	( )
5	**Pod apagado** sob carga constante: falhas e tempo de recuperação	`kubectl delete pod`	( )	( )
6	**Nó drenado** sob carga constante: falhas e tempo de recuperação	`kubectl drain` / `uncordon`	( )	( )
7	**Rolling update** sob carga: falhas e coexistência das versões	`kubectl set image` + `X-Versao`	( )	( )
8	`kubectl rollout undo` executado e comentado	`rollout history`	( )	( )
9	**Cache** (ou assíncrono): com × sem, mesma carga	2 resumos + gráfico	( )	( )
10	Cada experimento com hipótese, procedimento, evidências, tabela, gráfico e análise	rascunho do relatório	( )	( )
11	Resultados brutos de todas as execuções versionados	pasta `resultados/`	( )	( )

5. RESULTADOS (PREENCHIMENTO PELO GRUPO)

5.1 Escalabilidade horizontal manual – rota: _____ trabalhadores: ____ duração: ____ s

RÉPLICAS	VAZÃO (REQ/S)	P50 (MS)	P95 (MS)	ERROS (%)	GANHO MEDIDO	GANHO IDEAL
1					1,0	1
2						2
4						4
8						8

5.2 HPA – linha do tempo

EVENTO	HORA	RÉPLICAS	OBSERVAÇÃO (CPU, P95)
Início da carga (rampa)			
Primeira decisão de escala (`SuccessfulRescale`)			
Novos pods prontos (`1/1`)			
Latência p95 de volta ao patamar aceitável			
Fim da carga / início da redução			

5.3 Resiliência e 5.4 Rolling update

CENÁRIO	REQUISIÇÕES	FALHAS	TEMPO ATÉ POD SUBSTITUTO PRONTO	TEMPO ATÉ PATAMAR NORMAL	VERSÕES VISTAS
Pod apagado					—
Nó drenado					—
Rolling update			—		
Rollout undo			—		

5.5 Cache / processamento assíncrono – rota: _____ trabalhadores: ____

CENÁRIO	VAZÃO (REQ/S)	P50 (MS)	P95 (MS)	ERROS (%)	CPU DO BANCO
Sem cache / escrita síncrona					
Com cache / fila assíncrona					

6. VERIFICAÇÃO AO VIVO

Solicitada pelo professor durante a aula, com o gerador de carga rodando. Anotar o resultado observado.

AÇÃO	COMANDO	RESULTADO OBSERVADO
Escalar o serviço mais exigido de 1 para 4 réplicas e mostrar a vazão subir	`kubectl scale`	_____ → _____ req/s
Apagar um pod e contar as requisições que falharam	`kubectl delete pod`	_____ falhas
Publicar nova versão e mostrar as duas coexistindo	`kubectl set image`	
Explicar por que o ganho com 8 réplicas não foi 8×	—	

7. O QUE FOI CONSTRUÍDO ATÉ ESTA DATA / DIFICULDADES ENCONTRADAS

Descrever o que já funciona, o que falta e os problemas enfrentados.

8. VISTO DO PROFESSOR

Situação da etapa	( ) Adequada ( ) Adequada com ressalvas ( ) Insuficiente ( ) Não apresentada

Observações:

9. ASSINATURAS

Aluno(a) 1	Aluno(a) 2	Aluno(a) 3
Aluno(a) 4	Aluno(a) 5	Professor(a) supervisor(a)

ANEXO – GUIA RÁPIDO DOS EXPERIMENTOS DE ESCALABILIDADE E RESILIÊNCIA

Material de apoio ao preenchimento da ficha da **Etapa 5**. Detalhamento completo na **Aula Prática 5 – Experimentos de escalabilidade e resiliência**.

A. UM EXPERIMENTO, SEIS PARTES

PARTE	O QUE ESCREVER
Hipótese	O que o grupo espera, e por quê, **antes** de medir. Ex.: “com 4 réplicas a vazão quadruplica, porque o gargalo é a CPU do pod”.
Procedimento	Comandos e parâmetros exatos: réplicas, trabalhadores, duração, rota, repetições.
Evidências	Saídas de `kubectl` com hora, arquivos do gerador, eventos.
Tabela	Os números da seção 5 desta ficha.
Gráfico	Um por experimento; eixos com unidade; eventos marcados na linha do tempo.
Análise crítica	A hipótese se confirmou? Se não, **por quê**? Qual recurso limitou? O que mudaria o resultado?

B. LEI DE AMDAHL – POR QUE 8 RÉPLICAS NÃO DÃO 8×

Ganho(N) = 1 / (s + (1 − s) / N), em que **s** é a fração do trabalho que **não** se divide entre as réplicas (o banco único, a fila, o gateway, a CPU da máquina hospedeira). Com s = 10%, o ganho máximo, com infinitas réplicas, é 1 / 0,1 = **10×**; com 8 réplicas, 1 / (0,1 + 0,9/8) ≈ **4,7×**. Estimem o **s** do sistema a partir do ganho medido e digam qual componente ele representa.

C. AS PARCELAS DO ATRASO DO HPA

PARCELA	ORDEM DE GRANDEZA	DE ONDE VEM
Coleta de métricas	até ~15 s	O metrics-server lê a CPU dos pods periodicamente
Ciclo do HPA	até 15 s	O controlador recalcula as réplicas a cada 15 s
Agendamento e partida do contêiner	segundos	Scheduler escolhe o nó; imagem já presente; aplicação inicializa
Readiness	`periodSeconds`	O pod só entra no Service depois de passar na readiness
Propagação até o gateway	segundos	Ingress Controller atualiza a lista de destinos
Redução (*scale down*)	~5 min	Janela de estabilização: sobe rápido, desce devagar

D. FALHAS, DRENAGEM E ATUALIZAÇÃO – O QUE EXPLICAR

SITUAÇÃO	PONTO DE ANÁLISE
Pod apagado	Requisições em andamento no pod morrem; o Service só deixa de mandar tráfego quando o pod sai dos *endpoints*. A readiness decide quando o substituto entra.
Nó drenado	`drain` despeja os pods com aviso (encerramento gracioso). Pod com volume local (banco) pode ficar `Pending`: discutam. `PodDisruptionBudget` é o mecanismo que limita quantos pods saem de uma vez.
Rolling update	`maxSurge`/`maxUnavailable` controlam a troca; `preStop` dá tempo ao gateway de parar de enviar tráfego antes do SIGTERM. Comparem com e sem `preStop`.
Cache	Menos trabalho por requisição: o banco deixa de ser o gargalo. Custo: dado até *TTL* segundos desatualizado.